
Scalable Hyperparameter-Divergent Ensemble Training with Automatic Learning Rate Exploration for Large Models

Hailing Cheng
LinkedIn Inc.
Mountain View, CA 94043
haicheng@linkedin.com

Tao Huang
LinkedIn Inc.
Mountain View, CA 94043
taohuang@linkedin.com

Chen Zhu
LinkedIn Inc.
Mountain View, CA 94043
chzhu@linkedin.com

Antonio Alonso
LinkedIn Inc.
Mountain View, CA 94043
aalonso@linkedin.com

Abstract

Training large neural networks with data-parallel stochastic gradient descent allocates N GPU replicas to compute effectively identical updates—a practice that leaves the rich space of learning rate configurations entirely unexplored during training. We propose **Hyperparameter-Divergent Ensemble Training (HDET)**, a method that repurposes these replicas for simultaneous learning rate exploration at negligible communication overhead. HDET operates in alternating phases: a *fan-out* stage in which replicas train independently under a structured, symmetric spread of learning rates, and a *converge* stage in which parameters are averaged across all replicas via AllReduce every T steps. Building on this ensemble substrate, we further propose an **automatic learning rate (auto-LR) controller** that treats the relative training loss across replicas as a performance signal, updating the shared base schedule toward higher-performing configurations via a momentum-based gradient-free meta-update. The combined method produces a self-adapting learning rate schedule that improves both optimization quality and generalization without additional hyperparameter sweeps or training budget.

Crucially, the framework generalizes beyond learning rate: any scalar hyperparameter that does not alter model architecture—such as dropout rate, attention scale temperature, or weight-decay coefficient—can be explored across replicas using the same fan-out/converge protocol, with inter-replica loss differences serving as zero-order *hypergradients* that guide the search direction. HDET is implemented as a drop-in replacement for PyTorch’s OneCycleLR scheduler, requiring no changes to model architecture, optimizer, or data pipeline. Code is available at https://github.com/hailingc/ensemble_training.

1 Introduction

The learning rate schedule is among the most consequential hyperparameter choices in large-model training. Popular schedules—one-cycle annealing [Smith and Topin, 2019], cosine decay [Loshchilov and Hutter, 2017], and linear warmup-decay [Vaswani et al., 2017]—must be specified before training begins, leaving practitioners to rely on expensive grid searches or heuristics calibrated on smaller

proxy models. Even when a good schedule is found, it may not remain optimal as model or dataset scale changes across training runs.

Simultaneously, large-model training routinely parallelizes across hundreds of GPU replicas using data-parallel stochastic gradient descent (DP-SGD). In the standard paradigm, every replica maintains an identical copy of the model parameters and advances under an identical learning rate; the replica’s sole distinguishing role is the mini-batch it processes. Beyond improving gradient estimates through a larger effective batch, the additional hardware contributes *no qualitative diversity* to the optimization trajectory.

We observe that this uniformity is a missed opportunity. The N parallel replicas could explore N distinct learning rate trajectories simultaneously, at no additional hardware cost. Moreover, if those diverging trajectories periodically merge through parameter averaging, the resulting model can harvest ensemble-like generalization benefits within a *single* training run.

We formalize this intuition in **Hyperparameter-Divergent Ensemble Training (HDET)**, which interleaves two phases atop any existing DDP+OneCycleLR setup:

1. **Fan-out:** each of N GPU replicas trains under a distinct learning rate drawn from a symmetric spread $[\bar{\eta}(1 - \alpha), \bar{\eta}(1 + \alpha)]$ around the current base schedule $\bar{\eta}^{(t)}$, where $\alpha \geq 0$ is the spread ratio;
2. **Converge:** every T training steps, parameters are averaged across all replicas via AllReduce, producing a unified model that seeds the next fan-out round.

On top of this ensemble substrate we introduce an **auto-LR controller** that uses the relative training loss across replicas as a performance signal. Replicas with below-average loss receive proportionally higher weight in a softmax-weighted average of per-rank learning rates; the resulting performance deviation drives a momentum-based velocity that shifts the base schedule up or down at each sync point. If high-LR replicas consistently outperform low-LR ones, the base schedule adapts upward; if the opposite holds, it adapts downward. This constitutes a gradient-free, within-run optimizer over the learning rate schedule, eliminating the need for a priori schedule selection.

Contributions.

- We propose HDET, a training method that transforms existing DDP replicas into a structured learning rate exploration ensemble at zero hardware overhead (§3).
- We introduce an automatic LR controller that uses rank-wise loss signals to adaptively update the learning rate via a gradient-free momentum update, requiring no additional tuning (§3.4).
- We identify a practical two-stage strategy—warm noisy initialization from a pre-trained checkpoint followed by HDET ensemble training—that jointly maximizes model quality and training stability: warm initialization seeds each replica near a high-quality solution, while periodic weight averaging prevents divergence under aggressive learning rates that would destabilize either technique alone (§3.5, §4).
- We demonstrate empirically that HDET consistently improves final model quality and convergence speed on production-scale training tasks (§4).
- We release a reference implementation as a drop-in PyTorch scheduler at https://github.com/hailingc/ensemble_training.

2 Related Work

Learning-rate scheduling and online adaptation. Standard schedules—one-cycle [Smith and Topin, 2019], cyclical [Smith, 2017], cosine annealing [Loshchilov and Hutter, 2017], and warmup-decay [Vaswani et al., 2017]—must be fixed before training; OneCycleLR [Paszke et al., 2019] is the base scheduler HDET subclasses. Online methods eliminate this: Hypergradient Descent [Baydin et al., 2018], L4 [Rolinek and Martius, 2018], D-Adaptation [Defazio and Mishchenko, 2023], DoG [Ivgi et al., 2023], and Schedule-Free [Defazio et al., 2024] all derive step sizes from gradient statistics or loss dynamics. HDET differs: its adaptation signal is *cross-replica performance* rather than gradient information.

Learning-rate diversity and population-style exploration. Alrao [Blier et al., 2019] assigns different learning rates *within* a single network, covering a wide LR range at negligible cost. PBT [Jaderberg et al., 2017] maintains a population of models, periodically evaluating and copying hyperparameters from better to worse members. HDET shares concurrent hyperparameter exploration but reuses existing DDP replicas and merges them by AllReduce averaging rather than replacement, requiring no extra workers and preserving gradient information from every replica.

Distributed data-parallel training. Standard DP-SGD [Li et al., 2020] synchronizes gradients via AllReduce, keeping all ranks at identical parameters $\theta_r^{(t)} = \theta^{(t)}$. HDET retains gradient synchronization but assigns rank-specific LR multipliers, letting parameters diverge between sync intervals.

Weight averaging and ensemble merging. SWA [Izmailov et al., 2018], EMA [Ho et al., 2020], and Model Soups [Wortsman et al., 2022] average model weights sequentially or after training to reach flatter minima or improve generalization. SWAP [Gupta et al., 2020] extends SWA to parallel workers trained simultaneously, the setting most similar to HDET; however, SWAP workers share identical hyperparameters and differ only in effective batch size, whereas HDET deliberately assigns distinct learning rates across replicas to explore the hyperparameter space before each averaging step.

Worker divergence and periodic synchronization. EASGD [Zhang et al., 2015], Local SGD [Haddadpour et al., 2019], DiLoCo [Douillard et al., 2023], and GRAWA [Dimlioglu and Choromanska, 2024] all allow workers to drift from a shared solution between sync events to reduce communication or improve convergence. HDET differs: divergence is *intentionally induced* by a structured LR spread while standard synchronized-gradient DDP is retained throughout.

Gradient-free population optimization. NES [Wierstra et al., 2014] and ES [Salimans et al., 2017] optimize model parameters via a population of candidates and scalar fitness signals, requiring no gradient information. HDET’s auto-LR controller borrows this population-signal idea narrowly: inter-replica losses steer a single shared scalar (the base LR) while standard gradient-based weight updates remain intact.

Deep ensembles. Deep Ensembles [Lakshminarayanan et al., 2017] capture epistemic uncertainty by training N independent models from different initializations, at $N \times$ cost; diversity arises primarily from models reaching different loss basins [Fort et al., 2019]. HDET generates comparable diversity within a single run by using the LR spread to push replicas toward different basins before averaging.

3 Hyperparameter-Divergent Ensemble Training

3.1 Overview

HDET adds three targeted modifications to standard DDP+OneCycleLR training: (1) a structured LR spread across replicas at initialization; (2) periodic all-reduce weight averaging every T steps; and (3) an optional auto-LR controller that adjusts the base schedule between sync intervals. Figure 1 illustrates the full fan-out/converge cycle.

3.2 Learning Rate Divergence (Fan-Out)

For N ranks and spread ratio $\alpha \geq 0$, define $\delta = \max(\frac{N-1}{2}, 0.5)$ and assign each rank r the multiplier

$$\rho_r = 1 + \alpha \cdot \frac{r - \frac{N-1}{2}}{\delta}, \quad (1)$$

so that $\sum_r \rho_r = N$ (symmetric around 1) and $\rho_r \in [1-\alpha, 1+\alpha]$. Rank r trains under $\eta_r^{(t)} = \eta^{(t)} \cdot \rho_r$, where $\eta^{(t)}$ is the shared OneCycleLR base schedule, but all ranks share the same AllReduce gradient:

$$\theta_r^{(t+1)} = \theta_r^{(t)} - \eta_r^{(t)} \cdot \bar{g}^{(t)}. \quad (2)$$

Setting $\alpha=0$ recovers standard DDP exactly.

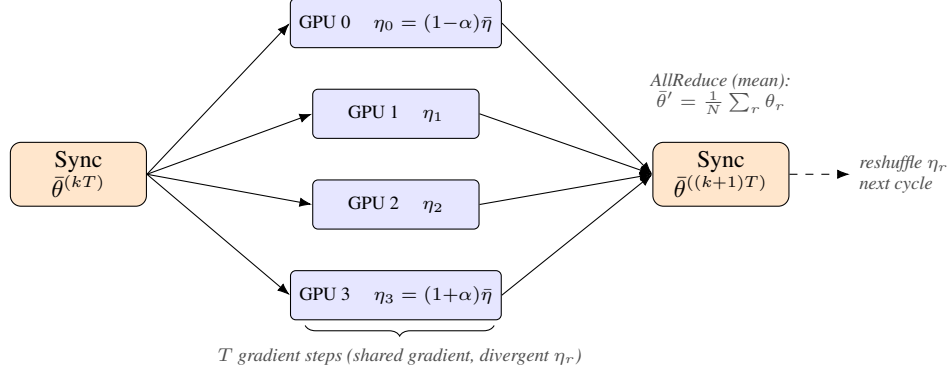


Figure 1: One HDET fan-out/converge cycle with $N=4$ replicas and spread ratio α . All replicas share the same all-reduced gradient at each step but take steps of different sizes, causing parameters to diverge. Every T steps an AllReduce average collapses the ensemble into a single model. Learning rates are randomly reassigned (reshuffled) before the next cycle.

3.3 Weight Averaging (Converge)

Every T steps, HDET AllReduce-averages all model parameters across ranks:

$$\theta_r^{(t)} \leftarrow \frac{1}{N} \sum_{r'=0}^{N-1} \theta_{r'}^{(t)}, \quad \forall r. \quad (3)$$

By SWA theory [Izmailov et al., 2018], averaged solutions from diverse trajectories occupy flatter minima and generalize better than any single trajectory. HDET produces this diversity explicitly in parallel rather than sequentially. The added communication cost over standard DDP is $O(1/T)$, negligible for $T \geq 100$. In our production experiments (8 H100 GPUs, $T=1,000$), this extra AllReduce adds approximately 0.1% wall-clock time.

3.4 Automatic Learning Rate Controller

After S_{warm} warmup steps, at each sync step k the controller computes per-rank losses $L_r^{(k)} = \frac{1}{T} \sum_t \ell(\theta_r^{(t)}, \mathcal{B}_r^{(t)})$, normalizes them against the mean $\bar{L}^{(k)}$ with temperature σ , and softmax-weights each rank:

$$s_r^{(k)} = \frac{\bar{L}^{(k)} - L_r^{(k)}}{\sigma}, \quad w_r^{(k)} = \frac{\exp(s_r^{(k)})}{\sum_{r'} \exp(s_{r'}^{(k)})}. \quad (4)$$

The deviation of the performance-weighted average LR from the unweighted mean is the *hypergradient* signal:

$$\tilde{\eta}^{(k)} = \sum_r w_r^{(k)} \eta_r^{(kT)}, \quad \bar{\eta}^{(k)} = \frac{1}{N} \sum_r \eta_r^{(kT)}, \quad (5)$$

$$\Delta^{(k)} = \tilde{\eta}^{(k)} - \bar{\eta}^{(k)}. \quad (6)$$

$\Delta > 0$ when higher-LR replicas outperform, signaling the base LR should rise. A momentum velocity smooths the update:

$$v^{(k+1)} = \beta v^{(k)} + (1 - \beta) \Delta^{(k)}, \quad \bar{\eta}^{(k+1)} = \bar{\eta}^{(k)}(1 - \gamma) + v^{(k+1)} \lambda. \quad (7)$$

where γ is set to match OneCycleLR’s decay floor over $K=(S-S_{\text{warm}})/T$ remaining sync intervals:

$$\gamma = 1 - \exp\left(-\frac{\log(f_{\text{div}}/f_{\text{final}})}{K}\right). \quad (8)$$

Random reassignment across hyperparameters. After updating $\bar{\eta}^{(k+1)}$, a fresh random permutation π re-assigns multipliers to ranks:

$$\eta_r^{(k+1)} = \bar{\eta}^{(k+1)} \cdot \rho_{\pi(r)}. \quad (9)$$

Algorithm 1 Hyperparameter-Divergent Ensemble Training (HDET)

Require: Model θ , optimizer, total steps S , world size N , OneCycleLR params $(\eta_{\max}, f_{\text{div}}, f_{\text{final}}, p)$, spread ratio α , sync interval T , warmup steps S_{warm} , auto-LR params (β, σ, λ)

```
1: Init:  $v \leftarrow 0$ ;  $L_{\text{acc}} \leftarrow 0$ 
2: Set  $\rho_r$  via Eq. (1); scale  $\text{base\_lrs} \leftarrow \text{base\_lrs} \times \rho_r$  on rank  $r$ 
3: for  $t = 1, \dots, S$  do
4:   Forward + backward; AllReduce gradients  $\bar{g}^{(t)}$  (standard DDP)
5:   Compute  $\eta_r^{(t)}$  from OneCycleLR formula with scaled  $\text{base\_lrs}$ 
6:   Update:  $\theta_r \leftarrow \theta_r - \eta_r^{(t)} \cdot \bar{g}^{(t)}$ 
7:    $L_{\text{acc}} \leftarrow L_{\text{acc}} + \ell(\theta_r, \mathcal{B}_r^{(t)})$ 
8:   if  $t \bmod T = 0$  then
9:     Converge: AllReduce  $\theta_r \leftarrow \frac{1}{N} \sum_{r'} \theta_{r'}$  (Eq. 3)
10:    if auto-LR enabled and  $t \geq S_{\text{warm}}$  then
11:       $L_r \leftarrow L_{\text{acc}}/T$ ;  $L_{\text{acc}} \leftarrow 0$ 
12:      AllGather  $\{L_r\}_{r=0}^{N-1}$ ; compute  $\{w_r\}$  via Eq. (4)
13:      AllGather  $\{\eta_r^{(t)}\}_{r=0}^{N-1}$ ; compute  $\tilde{\eta}, \bar{\eta}, \Delta$  via Eqs. (5)–(6)
14:       $v \leftarrow \beta v + (1 - \beta)\Delta$  (Eq. 7)
15:       $\bar{\eta} \leftarrow \bar{\eta}(1 - \gamma) + v \cdot \lambda$  (Eq. 7)
16:      Sample permutation  $\pi$  for random hyperparameter combinations; set  $\eta_r \leftarrow \bar{\eta} \cdot \rho_{\pi(r)}$ 
17:    else
18:      Update  $\eta_r^{(t)}$  using standard OneCycleLR formula
19:    end if
20:  end if
21: end for
22: return  $\theta_r$  (all ranks identical at last sync step)
```

This is critical when exploring *multiple* hyperparameters simultaneously (e.g. per-group LR_s). Fixed rank-to-value assignments would create spurious cross-parameter correlations that confound the hypergradient signal; drawing an independent permutation $\pi^{(j)}$ per hyperparameter j decorrelates them, allowing the controller to track whichever hyperparameter most explains rank-loss variance at each sync. In this paper we demonstrate simultaneous exploration across four optimizer parameter groups (embedding, no_decay, non_embedding, transformer), whose emergent differential decay rates are analyzed in §5.

3.5 Warm Noisy Initialization

HDET can optionally be seeded from a pre-trained checkpoint rather than from random initialization. Before the first fan-out phase, independent Gaussian noise is added to each parameter:

$$\theta_r \leftarrow \bar{\theta}_0 + \epsilon_r, \quad \epsilon_r \sim \mathcal{N}\left(0, \frac{\nu^2 \|\bar{\theta}_0\|^2}{d}\right), \quad (10)$$

where $\bar{\theta}_0$ is the pre-trained checkpoint, d is the total number of parameters, and ν controls the perturbation scale (we use $\nu=0.01$ throughout). This initializes each replica in a distinct neighborhood of a high-quality solution, providing a warm start that accelerates early convergence and encourages replicas to explore different regions of the loss landscape near the checkpoint.

Warm initialization *alone* is insufficient to stabilize training at aggressive learning rates: without periodic weight averaging, noisy weights compound with high-LR updates and eventually diverge (the **Warm-Init** ablation in §4 confirms this). Its role is to supply better-quality starting points for the ensemble, not to replace the stabilization provided by the fan-out/converge cycle. When HDET weight averaging is active, warm-init and the auto-LR controller each contribute independently, as the ablation results demonstrate.

Remark 1 (Generalization to arbitrary hyperparameters). *Algorithm 1 is presented for learning rate η but applies to any scalar hyperparameter h satisfying: (i) varying h across replicas does not change model architecture (so AllReduce averaging remains well-defined), and (ii) h influences*

Table 1: Training loss on production news-feed data. Lower is better. † indicates training crash (loss grew unboundedly). All HDET variants use $\eta_{\max}=0.0009$, which is $9\times$ higher than the conservative baseline.

Model	Description	Avg LR	Train Loss
Baseline-Low	Standard DDP, $\alpha=0$	0.0001	3.294
Baseline-High	Standard DDP, $\alpha=0$	0.0009	4.169†
Warm-Init	Perturbed init, $\alpha=0$	0.0009	4.674†
HDET w/o auto-LR	LR spread + warm init, no auto-LR	0.0009	3.280
HDET w/o warm init	LR spread + auto-LR, cold start	0.0009	3.281
HDET (full)	LR spread + warm init + auto-LR	0.0009	3.277

training loss differentially (so loss comparisons carry signal about h 's optimal value). Candidates include dropout rate, label-smoothing coefficient, weight decay, and attention temperature. In each case $\Delta = \bar{h} - h$ is a zero-order hypergradient estimating $-\partial\mathcal{L}/\partial h$ from inter-replica loss differences, without back-propagating through the training loop. Multiple hyperparameters can be co-explored simultaneously by assigning each replica an independent spread vector, with a separate random permutation per hyperparameter to prevent cross-parameter correlation.

4 Experiments

4.1 Experimental Setup

Task and data. We evaluate on a production-scale news-feed dataset from a major social network, comprising one year of user-item interaction logs. Models are trained jointly on three engagement tasks (**Contribution**, **Like**, **LongDwell**) for **one epoch**—standard in production settings where the data stream is non-repeating and any change in training loss directly reflects optimization quality.

Backbone. All experiments share the same fixed **AttnMVP** backbone [Cheng, 2026] with identical architecture, optimizer (Adam), and feature set. Backbone details are orthogonal to HDET and are reported in Cheng [2026]; our focus is solely on the training dynamics.

HDET configuration. All variants run on **8 NVIDIA H100 GPUs** ($N=8$); each single-epoch run takes ≈ 30 hours, constraining ablation breadth. Spread $\alpha=1/9$, giving multipliers spanning $[0.0008, 0.001]$ with mean 0.0009 (matching the baseline). $T=1,000$ steps; $S_{\text{warm}}=100,000$ steps; auto-LR: $\beta=0.9$, $\sigma=0.1$, $\lambda=0.5$ (fixed, not tuned).

Ablation configurations. We compare six configurations that isolate each HDET component:

1. **Baseline-Low** ($\eta=0.0001$, no spread): conservative-LR reference.
2. **Baseline-High** ($\eta=0.0009$, no spread): high-LR without ensemble stabilization.
3. **Warm-Init** ($\eta=0.0009$, no spread): warm noisy initialization from a pre-trained checkpoint (§3.5), no weight averaging. Tests whether initialization alone is sufficient for high-LR stability.
4. **HDET w/o auto-LR** ($\eta=0.0009$, spread + warm init): fan-out/converge with LR spread and weight averaging; auto-LR disabled.
5. **HDET w/o warm init** ($\eta=0.0009$, spread + auto-LR): full HDET from a cold start; ablates warm initialization.
6. **HDET (full)** ($\eta=0.0009$, spread + warm init + auto-LR): complete method.

We report average training loss across the three tasks (lower is better); † denotes training crash.

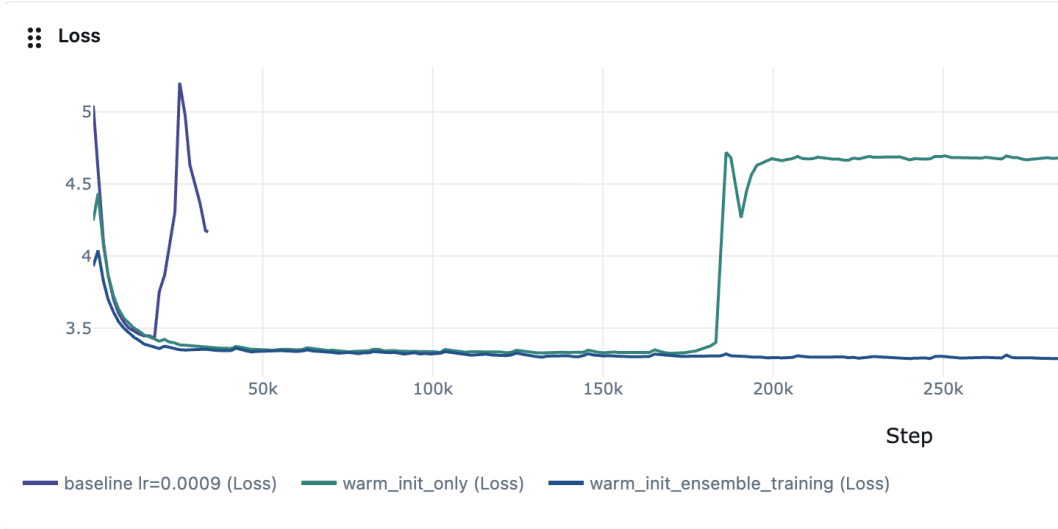


Figure 2: Training loss vs. steps at $\eta_{\max}=0.0009$. **Baseline** (gray) crashes at $\approx 18\text{K}$ steps, stabilizing at 4.169. **Warm-Init** (teal) survives to $\approx 185\text{K}$ steps then diverges (4.674): warm initialization extends stability but cannot replace weight averaging. **HDET** (dark blue) converges smoothly to 3.277—below Baseline-Low (3.294, not shown) trained at $9\times$ lower LR.

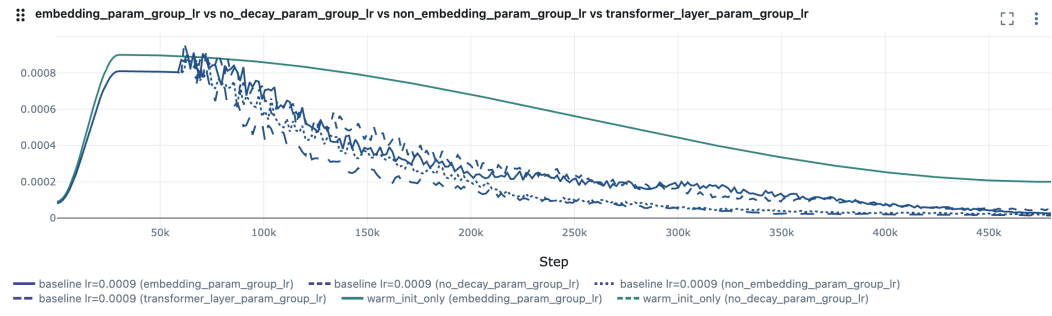


Figure 3: Per-group LR schedules for four parameter groups. Solid: baseline (OneCycleLR, $\eta_{\max}=0.0009$). Dashed: HDET GPU 0 (lowest-LR replica, $\rho_0 \cdot 0.0009=0.0008$); the 8-replica mean equals the 0.0009 baseline. Periodic fluctuations mark each sync interval where the auto-LR controller updates the base schedule.

4.2 Main Results

4.3 Analysis

Stability (Figure 2). Both Baseline-High and Warm-Init diverge at $\eta=0.0009$; HDET trains stably throughout. Warm-Init diverges *more* severely than Baseline-High (4.674 vs. 4.169), confirming that weight averaging—not initialization—is the primary stabilizer.

Quality and ablation. HDET w/o auto-LR (3.280) already beats Baseline-Low (3.294) at $9\times$ higher LR, showing that LR spread + weight averaging alone discovers better optima. The two ablations—w/o auto-LR (3.280) and w/o warm init (3.281)—each contribute independently and comparably (≈ 0.013 – 0.014 each), and HDET (full, 3.277) combines both for the best result. The auto-LR gain is modest here because the initial schedule is already near-optimal; larger gains are expected when the hand-tuned baseline is farther from optimal.

Statistical reliability. We repeated training over **10+ independent models**; HDET consistently outperformed Baseline-Low on offline AUC and NE across all replications. The model is deployed in the production ranking system.

Per-group LR schedules (Figure 3). Figure 3 shows GPU 0 (peak LR $0.0008 = \rho_0 \cdot 0.0009$, the lowest-LR replica) for clarity; the 8-replica mean equals the 0.0009 baseline. The auto-LR controller produces periodic fluctuations at each sync and drives all groups to a residual LR of ≈ 0.0002 rather than near-zero, reflecting its discovery that a non-trivial final LR benefits this task.

5 Discussion

Exploration-exploitation in hyperparameter space. HDET implements a principled exploration-exploitation trade-off directly in hyperparameter space. The *fan-out* phase is the exploration step: replicas diverge under a structured spread of learning rates, simultaneously probing distinct regions of the loss landscape that no single schedule could visit concurrently. The *converge* phase is the exploitation step: AllReduce averaging collapses the ensemble into a single model that harvests the best of all explored trajectories, re-anchoring parameters near the consensus of the full replica population. Because the LR spread always keeps some replicas conservative, this consensus is guaranteed to lie in a stable region—which is why periodic averaging prevents divergence where a single aggressive LR cannot. Warm-Init without HDET illustrates the same mechanism in reverse: with no converge phase to re-anchor parameters, high-LR updates on perturbed weights compound unchecked until irrecoverable divergence at $\sim 185K$ steps.

Emergent differential LR decay. The auto-LR controller discovers distinct per-group decay rates without manual specification (Figure 3): *transformer* decays fastest (dense high-variance gradients converge quickly), *non_decay* parameters slowest (flat landscapes absorb gradient signal throughout), with embeddings and non-embedding layers in between. This ordering mirrors discriminative fine-tuning [Howard and Ruder, 2018] but emerges *automatically* from cross-replica loss signals.

Limitations. HDET’s auto-LR adapts per-group schedules from model performance across replicas, complementing Adam’s [Kingma and Ba, 2014] per-parameter gradient-moment adaptation; combining them is a natural next step. The three additional hyperparameters (α , T , σ) were fixed without tuning and consistently outperformed the baseline, though their interaction and architecture-dependence warrant further study.

6 Conclusion and Future Work

HDET converts the wasted uniformity of DDP replicas into a structured LR exploration ensemble via a fan-out/converge cycle ($O(1/T)$ extra communication), with an auto-LR controller that adapts the base schedule online from cross-replica loss signals and warm noisy initialization that seeds training near a high-quality checkpoint. On a production recommendation model, HDET trains stably with much higher LR than the LR that causes divergence in standard DDP and achieves a lower final loss (3.277 vs. 3.294), while its auto-LR controller autonomously discovers the empirically correct per-group decay ordering (transformer fastest, non-decay slowest).

Future directions. (i) *General hyperparameter exploration.* As formalized in Remark 1, any scalar hyperparameter that does not change model architecture (dropout, weight decay, temperature) can be substituted for LR, turning the cluster into a zero-cost hyperparameter search engine using inter-replica losses as hypergradients. (ii) *Group-aware initialization.* The emergent decay ordering provides an empirical prior for initializing per-group spread ratios, potentially accelerating group-wise schedule discovery. (iii) *LLM pretraining.* Extending HDET to multi-epoch pretraining over hundreds of billions of tokens offers more sync intervals for the auto-LR to adapt, with warm-init generalized to early-stage cold starts. (iv) *Convergence theory.* A formal analysis characterizing the interplay of LR spread, sync interval, and averaging on convergence and implicit regularization remains an open question.

Acknowledgments and Disclosure of Funding

The authors thank colleagues on the production ranking team for helpful discussions and infrastructure support. We also thank the anonymous reviewers for their constructive feedback.

References

References

- A. G. Baydin, R. Cornish, D. M. Rubio, M. Schmidt, and F. Wood. Online learning rate adaptation with hypergradient descent. In *International Conference on Learning Representations (ICLR)*, 2018.
- L. Blier, P. Wolinski, and Y. Ollivier. Learning with random learning rates. *arXiv preprint arXiv:1810.01322*, 2019.
- H. Cheng. Beyond interleaving: Causal attention reformulations for generative recommender systems, 2026. URL <https://arxiv.org/abs/2603.10369>.
- A. Defazio and K. Mishchenko. Learning-rate-free learning by D-adaptation. In *International Conference on Machine Learning (ICML)*, 2023.
- A. Defazio, X. A. Yang, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky. The road less scheduled. *arXiv preprint arXiv:2405.15682*, 2024.
- T. Dimlioglu and A. Choromanska. GRAWA: Gradient-based weighted averaging for distributed training of deep learning models. *arXiv preprint arXiv:2403.04206*, 2024.
- A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Szegedy, A. Sessa, A. Galashov, and A. T. Cemgil. DiLoCo: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023.
- S. Fort, H. Hu, and B. Lakshminarayanan. Deep ensembles: A loss landscape perspective. In *arXiv preprint arXiv:1912.02757*, 2019.
- V. Gupta, S. A. Serrano, and D. DeCoste. Stochastic weight averaging in parallel: Large-batch training that generalizes well. In *International Conference on Learning Representations (ICLR)*, 2020.
- F. Haddadpour, M. M. Kamani, M. Mahdavi, and V. R. Cadambe. Local SGD with periodic averaging: Tighter analysis and adaptive synchronization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.
- J. Howard and S. Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 328–339, 2018.
- M. Ivgi, O. Hinder, and Y. Carmon. DoG is SGD’s best friend: A parameter-free dynamic step size schedule. *arXiv preprint arXiv:2302.12022*, 2023.
- P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson. Averaging weights leads to wider optima and better generalization. In *Uncertainty in Artificial Intelligence (UAI)*, 2018.
- M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, et al. Population based training of neural networks. In *arXiv preprint arXiv:1711.09846*, 2017.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

- B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, et al. PyTorch distributed: Experiences on accelerating data parallel training. In *Proceedings of the VLDB Endowment*, volume 13, pages 3005–3018, 2020.
- I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- M. Rolinek and G. Martius. L4: Practical loss-based stepsize adaptation for deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- L. N. Smith. Cyclical learning rates for training neural networks. In *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 464–472. IEEE, 2017.
- L. N. Smith and N. Topin. Super-convergence: Very fast training of neural networks using large learning rates. *Proceedings of SPIE*, 11006:1100612, 2019.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- D. Wierstra, T. Forster, J. Peters, and J. Schmidhuber. Natural evolution strategies. In *Journal of Machine Learning Research*, volume 15, pages 949–980, 2014.
- M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carlin, S. Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, pages 23965–23998, 2022.
- S. Zhang, A. E. Choromanska, and Y. LeCun. Deep learning with elastic averaging SGD. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.